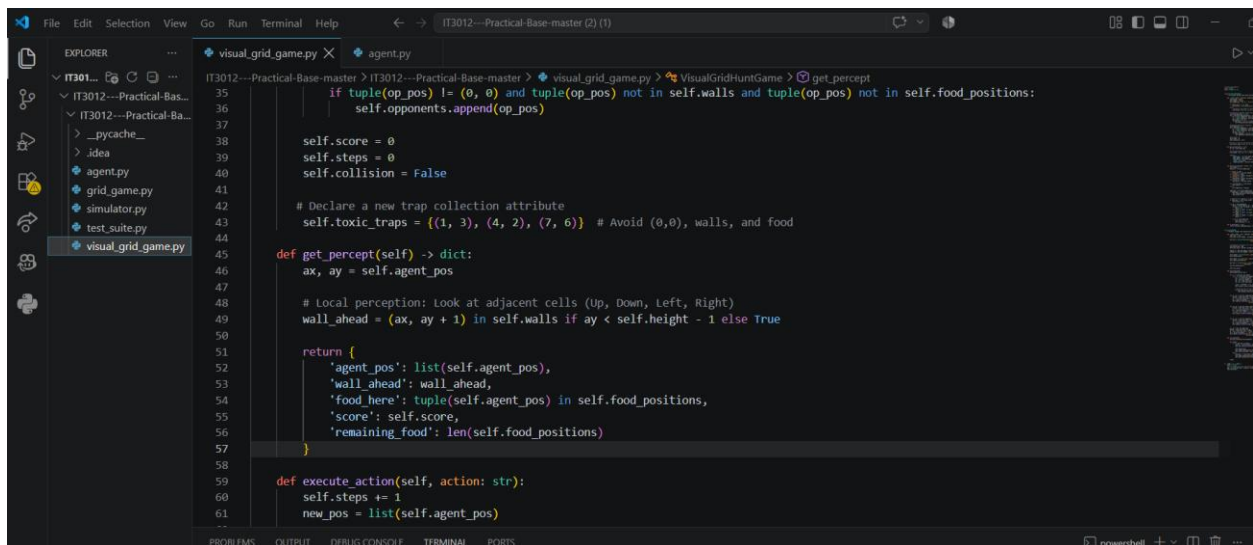


Part 1: Practical Implementation — Coding the Architectures

Step

1.1: Setting the Trap (Partial Observability)

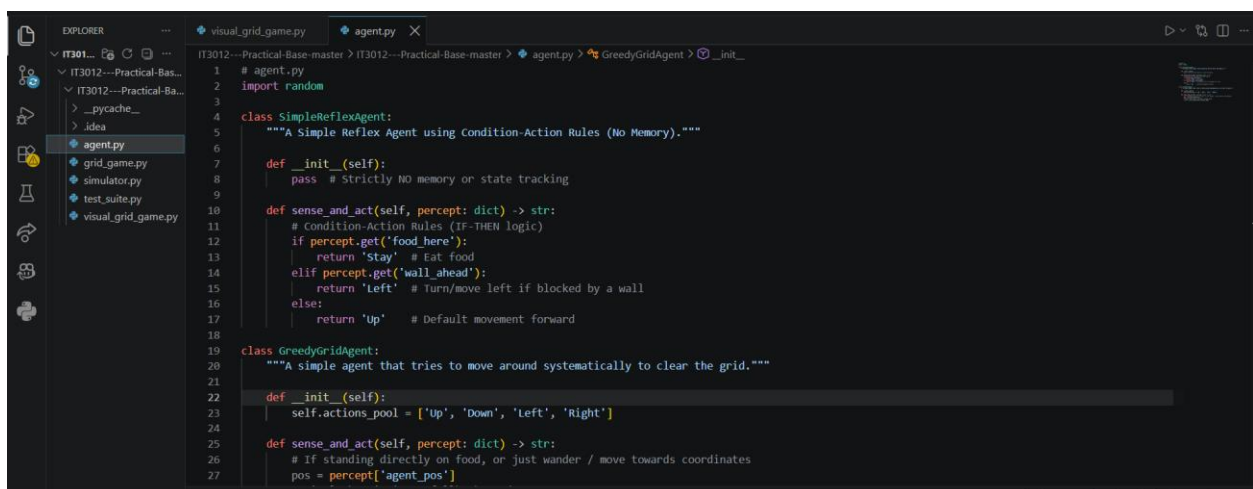


```
File Edit Selection View Go Run Terminal Help
IT3012---Practical-Base-master (2) (1)

EXPLORER
IT3012---Practical-Base-master
  IT3012---Practical-Bas...
    > __pycache__
    > idea
    agent.py
    grid_game.py
    simulator.py
    test_suite.py
    visual_grid_game.py

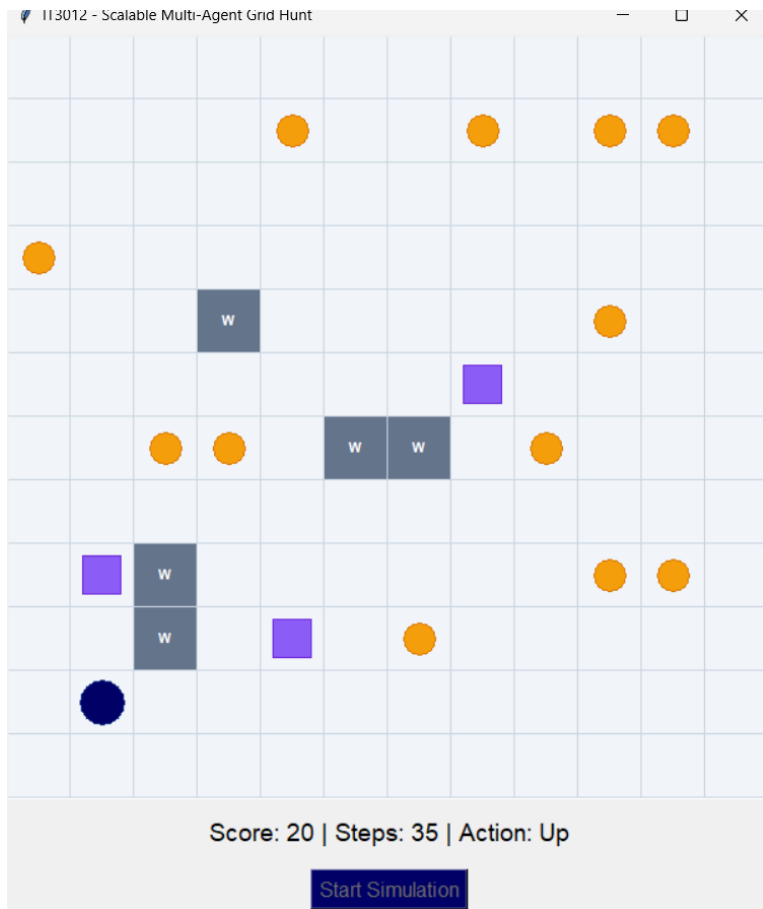
visual_grid_game.py
35
36     if tuple(op_pos) != (0, 0) and tuple(op_pos) not in self.walls and tuple(op_pos) not in self.food_positions:
37         self.opponents.append(op_pos)
38
39     self.score = 0
40     self.steps = 0
41     self.collison = False
42
43     # Declare a new trap collection attribute
44     self.toxic_traps = ((1, 3), (4, 2), (7, 6)) # Avoid (0,0), walls, and food
45
46     def get_percept(self) -> dict:
47         ax, ay = self.agent_pos
48
49         # Local perception: Look at adjacent cells (Up, Down, Left, Right)
50         wall_ahead = (ax, ay + 1) in self.walls if ay < self.height - 1 else True
51
52         return {
53             'agent_pos': list(self.agent_pos),
54             'wall_ahead': wall_ahead,
55             'food_here': tuple(self.agent_pos) in self.food_positions,
56             'score': self.score,
57             'remaining_food': len(self.food_positions)
58         }
59
60     def execute_action(self, action: str):
61         self.steps += 1
62         new_pos = list(self.agent_pos)
```

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

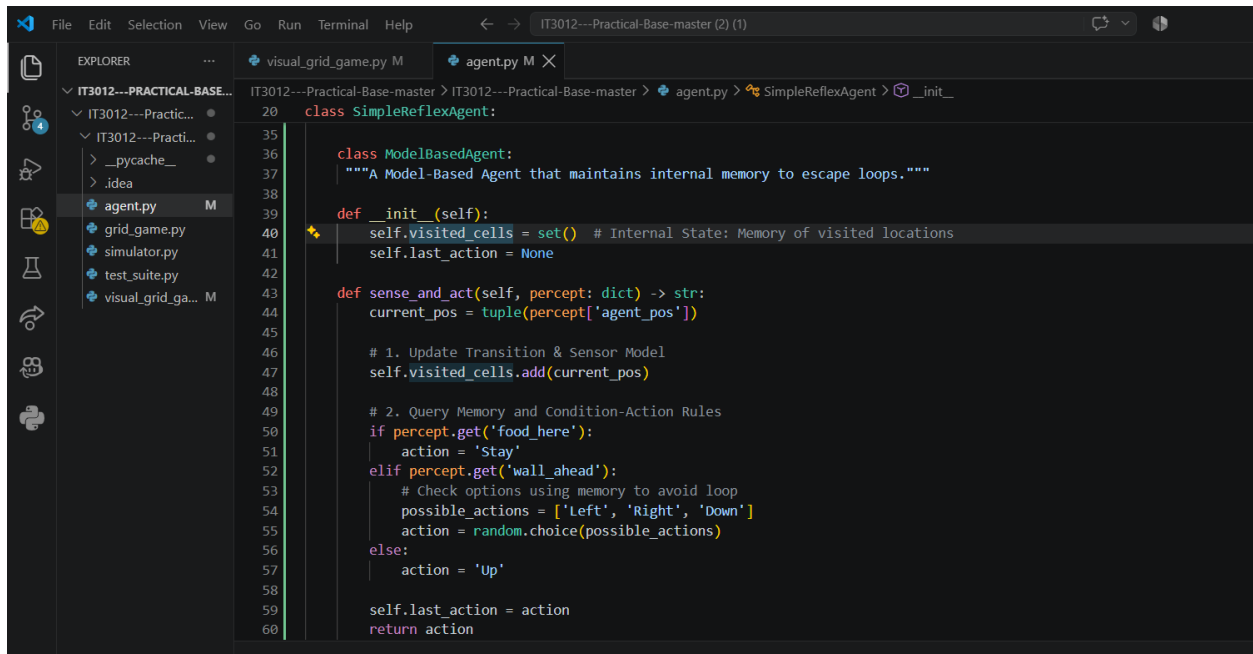


```
EXPLORER
IT3012---Practical-Base-master
  IT3012---Practical-Bas...
    > __pycache__
    > idea
    agent.py
    grid_game.py
    simulator.py
    test_suite.py
    visual_grid_game.py

agent.py
1 # agent.py
2 import random
3
4 class SimpleReflexAgent:
5     """A Simple Reflex Agent using Condition-Action Rules (No Memory)."""
6
7     def __init__(self):
8         pass # Strictly NO memory or state tracking
9
10    def sense_and_act(self, percept: dict) -> str:
11        # Condition-Action Rules (IF-THEN logic)
12        if percept.get('food_here'):
13            return 'Stay' # Eat food
14        elif percept.get('wall_ahead'):
15            return 'Left' # Turn/move left if blocked by a wall
16        else:
17            return 'Up' # Default movement forward
18
19 class GreedyGridAgent:
20     """A simple agent that tries to move around systematically to clear the grid."""
21
22     def __init__(self):
23         self.actions_pool = ['Up', 'Down', 'Left', 'Right']
24
25     def sense_and_act(self, percept: dict) -> str:
26        # If standing directly on food, or just wander / move towards coordinates
27        pos = percept['agent_pos']
```



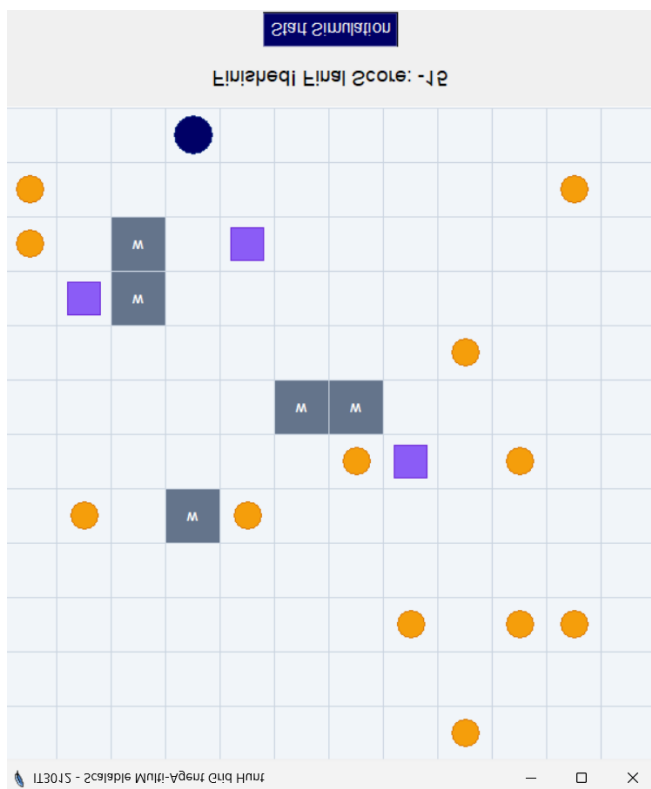
Step 1.3: The Model-Based Agent (Memory & State)



```
File Edit Selection View Go Run Terminal Help
IT3012---Practical-Base-master (2) (1)

EXPLORER
IT3012---Practical-Base-master
  IT3012---Practical-Base-master
    __pycache__
    .idea
    agent.py M
    grid_game.py
    simulator.py
    test_suite.py
    visual_grid_game.py M

IT3012---Practical-Base-master > IT3012---Practical-Base-master > agent.py > SimpleReflexAgent > __init__
20 class SimpleReflexAgent:
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36 class ModelBasedAgent:
37     """A Model-Based Agent that maintains internal memory to escape loops."""
38
39
40 def __init__(self):
41     self.visited_cells = set() # Internal State: Memory of visited locations
42     self.last_action = None
43
44 def sense_and_act(self, percept: dict) -> str:
45     current_pos = tuple(percept['agent_pos'])
46
47     # 1. Update Transition & Sensor Model
48     self.visited_cells.add(current_pos)
49
50     # 2. Query Memory and Condition-Action Rules
51     if percept.get('food_here'):
52         action = 'Stay'
53     elif percept.get('wall_ahead'):
54         # Check options using memory to avoid loop
55         possible_actions = ['Left', 'Right', 'Down']
56         action = random.choice(possible_actions)
57     else:
58         action = 'Up'
59
60     self.last_action = action
61     return action
```



Q1: According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex

environments like Chess? What happens as the agent's lifetime increases?

Answer: A Table-Driven Agent explicitly stores a lookup table mapping every possible percept sequence to an action. In complex environments, the number of possible percept sequences grows exponentially over time (P^t). This leads to a combinatorial explosion, making the table lookup size infinitely huge, requiring impossible amounts of storage and computational lookup time.

Q2: Look at the code you wrote for your SimpleReflexAgent. Identify and explain the specific lines of code that represent the "Condition-Action Rules".

Answer:

In SimpleReflexAgent.sense_and_act(), the lines:

Python

```
if percept.get('food_here'): return 'Stay'  
elif percept.get('wall_ahead'): return 'Left'  
else: return 'Up'
```

represent the Condition-Action Rules. They directly map current local perceptions (e.g., wall_ahead) to an immediate actuator movement ('Left') without consulting history.

Q3: Analyze exactly why the SimpleReflexAgent gets stuck in an infinite loop. How did "Partial Observability" and a lack of "Percept History" cause this failure?

Answer: Under partial observability, the agent only sees immediate adjacent cells (wall_ahead) and lacks full map awareness. Because it has no percept history/memory, every time it faces a wall, it evaluates the exact same condition and executes the exact same action ('Left'). Since the state doesn't break automatically, it loops infinitely between the same set of grid cells.

Q4: Evaluate how your ModelBasedAgent code handles the "Transition Model" and "Sensor Model".

Answer:

- Sensor Model: Handled when reading percept['agent_pos'] to determine the current state from raw percepts.
- Transition Model: Handled by self.visited_cells.add(current_pos) and tracking self.last_action, which updates internal memory about how actions alter position over time. This internal state allows the rule base to choose unvisited/alternative paths and break infinite loops.